Universidad Católica Andrés Bello

Escuela de Ingeniería Informática

Inteligencia Artificial: Aprendizaje Automático

Prof. Gustavo La Fontaine

INFORME EJECUTIVO DE PROYECTO

Sistema "Deep Retention" - Motor Predictivo de Retención Selectiva de Clientes

Equipo

Patricia Docasar | C.I. 30.330.091	Daniel Perez | C.I. 30.714.892
Andrés Valdivieso | C.I. 31.045.690	Salomon Trujillo | C.I. 30.125.275
Violeta Semprúm | C.I. 31.423.116	César Reyes | C.I. 31.101.802

Caracas, 12 de noviembre de 2026.

1. Resumen de Impacto Financiero y Retorno

- **Volumen Total de Clientes Analizados:** 10,000
- **Monto Total de Capital en Riesgo Crítico:** $105.06 millones
- **Capital Rescatable Estimado:** $41.30 millones
- **Inversión Total en Campañas de Fidelización:** $147,670
- **Retorno Sobre la Inversión (ROI):** 320%

Lectura de las cifras: el capital en riesgo crítico es la suma de saldos de los 1,211 clientes con probabilidad calibrada de fuga ≥ 0.5. El capital rescatable es el saldo que se espera conservar tras aplicar los incentivos; no es ganancia del banco. La rentabilidad se mide con la ganancia neta esperada, no con el saldo. El sistema recomienda intervenir a 1,735 clientes y deja 547 como "Salida Permitida".

Cálculo del ROI: ROI = Ganancia neta esperada / Inversión = $471,839 / $147,670 ≈ 320%. La ganancia neta ya descuenta el costo de cada incentivo. Las cifras de la cartera completa incluyen a los clientes con los que se entrenó la red. Como control, sobre el conjunto de prueba (1,500 clientes no vistos en el entrenamiento) el ROI es de 318%, con una inversión de $23,550 y una ganancia neta de $74,824.

2. Arquitectura de la Solución y Diagnóstico de Fuga

Configuración de la Red Neuronal: Arquitectura de Perceptrón Multicapa (MLP) secuencial estructurada en dos capas ocultas densas (128 y 64 neuronas, respectivamente) operando con funciones de activación ReLU. Para prevenir el sobreajuste y garantizar la estabilidad del entrenamiento, se incorporaron técnicas de regularización mediante Batch Normalization y Dropout (tasas de 0.3 y 0.2). La capa de salida emplea una única neurona con función de activación Sigmoide para emitir la probabilidad de deserción del cliente, optimizando los pesos a través del algoritmo Adam y utilizando la Entropía Cruzada Binaria (Binary Cross-Entropy) como función de pérdida.

Variables del modelo: el modelo usa 13 variables. Diez numéricas: CreditScore, Age, Tenure, Balance, NumOfProducts, HasCrCard, IsActiveMember, EstimatedSalary, Satisfaction Score y Point Earned. Tres categóricas: Geography, Gender y Card Type. La variable objetivo es Exited.

Preparación de datos: las variables numéricas se escalaron con StandardScaler y las categóricas se codificaron con One-Hot Encoding, ambas dentro de un ColumnTransformer

ajustado solo con el conjunto de entrenamiento y guardado en preprocessor.joblib. La partición fue estratificada por la variable objetivo: 70% entrenamiento, 15% validación y 15% prueba. Las entradas nuevas se validan (columnas y rangos) antes de predecir.

Prevención de fuga de datos: se excluyeron RowNumber, Surname y Complain. CustomerId se usa solo como identificador del cliente y no entra al modelo. Complain se descartó porque 2,034 de los 2,038 clientes que se fueron tienen una queja registrada: es una consecuencia de la salida que no estaría disponible al momento de predecir. Con ella, el ROC-AUC sería de aproximadamente 0.999, un resultado artificial.

Tratamiento del desbalance y del sobreajuste: la clase de fuga representa el 20.4% de los registros, por lo que se usó class_weight="balanced" (peso 2.45 para la clase de fuga y 0.63 para la clase estable). Además del Dropout, se aplicó Early Stopping sobre val_loss con paciencia de 15 épocas: el entrenamiento se detuvo en la época 43 y se restauraron los pesos de la mejor época (28). Como los pesos de clase inflan las probabilidades, estas se calibraron con escalado de Platt (regresión logística) sobre el conjunto de validación (calibrator.joblib). La calibración redujo el Brier Score de 0.147 a 0.106 y el ECE de 0.158 a 0.019; tras ella, la probabilidad media predicha (21.0%) coincide con la tasa real de fuga (20.4%).

Métricas de Diagnóstico Técnico:

ROC-AUC	Precision-Recall AUC	Error de Calibración (Brier Score)
0.851	0.685	0.106

Métricas calculadas sobre el conjunto de prueba (1,500 clientes). El ROC-AUC y el PR-AUC miden la capacidad del modelo para ordenar a los clientes por riesgo (el PR-AUC de un modelo aleatorio sería ≈ 0.20). El Brier Score (0.106) y el error de calibración esperado, ECE (0.019, calculado con 10 intervalos de probabilidad), miden qué tan fieles son las probabilidades: cuanto menores, mejor.

Con un umbral de 0.5, el modelo detecta el 74.2% de los clientes que se fugan (recall de 0.742), con una precisión de 0.484, un F1 de 0.586 y una exactitud global de 78.6%. De los 306 clientes que se fugaron en prueba, identificó 227 y dejó escapar 79; además, marcó como riesgo a 242 de los 1,194 clientes estables. En retención, este balance es aceptable: no detectar a un cliente que se va cuesta más que ofrecer un incentivo de bajo costo a uno que se habría quedado, y la regla financiera filtra después los casos no rentables.

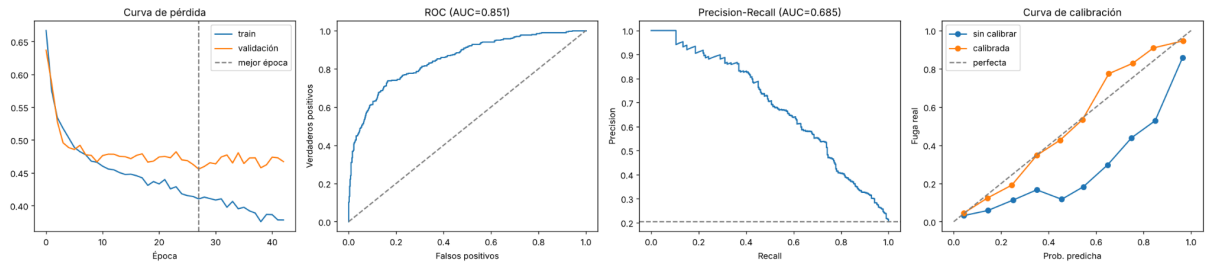


Figura 1. Curva de pérdida (entrenamiento y validación), curvas ROC y Precision-Recall, y curva de calibración (estas tres últimas, sobre el conjunto de prueba).

Factores Determinantes de Abandono:

- **Edad del cliente (Age):** es el factor con mayor impacto en el modelo (importancia SHAP media de 0.094). Los clientes de mayor edad empujan la probabilidad de fuga hacia arriba, mientras que los más jóvenes la reducen.
- **Cantidad de productos vinculados (NumOfProducts):** tiene un peso casi idéntico al de la edad (0.092). Los clientes con un solo producto tienen más riesgo que los que tienen dos, y el efecto más fuerte de todo el modelo lo producen los clientes con 3 o 4 productos, que elevan mucho la probabilidad de fuga.
- **Nivel de actividad operativa (IsActiveMember):** es el tercer factor (0.049). Los clientes inactivos tienen una probabilidad de fuga mayor que los activos, lo que convierte la reactivación en una palanca directa de retención.
- **País (Geography):** es el cuarto factor (0.041, agrupando sus categorías). Ser cliente en Alemania aumenta la probabilidad de fuga, mientras que estar en Francia o España la reduce ligeramente.
- **Otros factores:** un saldo más alto (Balance, 0.030) eleva ligeramente el riesgo, y las mujeres muestran una propensión a la fuga algo mayor que los hombres (Gender, 0.028). El resto de las variables tiene un impacto menor a 0.03.

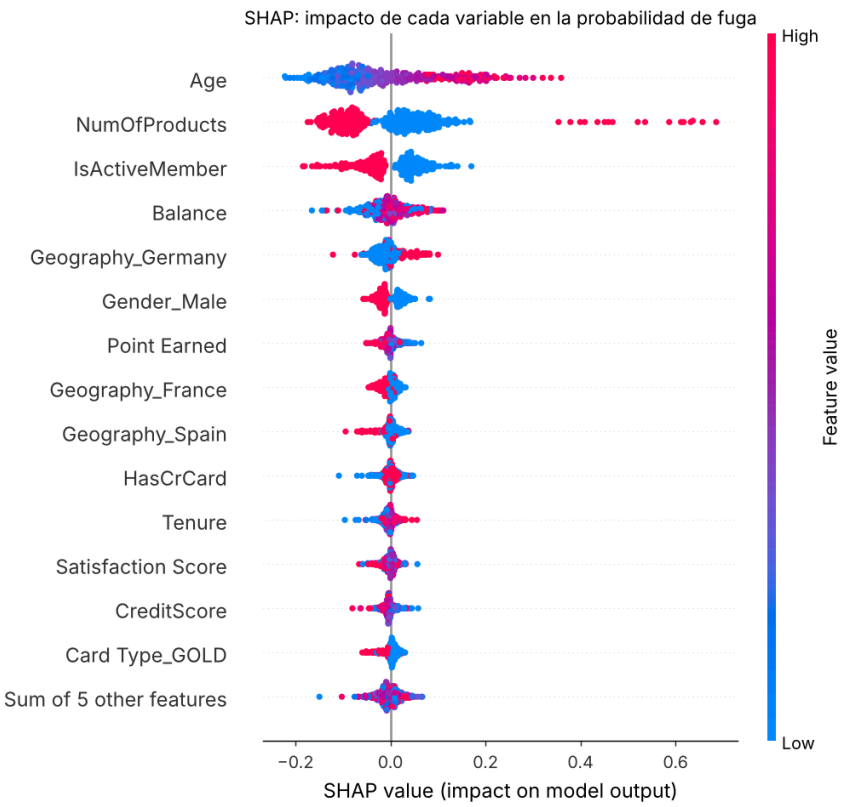


Figura 2. Impacto de cada variable en la probabilidad de fuga (SHAP). Cada punto es un cliente; el color indica el valor de la variable.

3. Matriz de Decisión Comercial y Rentabilidad

La decisión no se toma con un corte fijo de probabilidad, sino comparando la ganancia neta esperada de intervenir con su costo. Para cada cliente i y cada incentivo k:

$$\mathbf{E[G_i\square] = p_i \cdot e\square \cdot V_i - c\square}$$

donde p_i es la probabilidad calibrada de fuga, $e\square$ la efectividad del incentivo (reducción esperada de la probabilidad de fuga), V_i el margen anual del cliente y $c\square$ el costo del incentivo. El sistema interviene solo si $E[G_i\square] > 0$ y, entre los incentivos elegibles, elige el de mayor ganancia. De la ecuación se deduce el umbral de probabilidad de cada cliente:

$$\mathbf{p_i^* = c\square / (e\square \cdot V_i)}$$

Así, el umbral es menor para los clientes de alto margen y mayor para los de bajo margen. Si $V_i \leq 0$ (cliente deficitario), ninguna probabilidad justifica la inversión y el cliente queda como "Salida Permitida". Por política conservadora, los clientes con $p_i < 0.30$ se consideran estables y no reciben incentivo aunque la fórmula resulte positiva.

Margen del cliente: como el dataset no incluye comisiones ni volumen transaccional, el margen anual se estima con supuestos ajustables en la aplicación:

$$V_i = 1.5\% \times \text{Saldo} + \$60 \times \text{Productos} + \$30 \times \text{Tarjeta de crédito} - \$120$$

El modelo financiero incluye un parámetro de horizonte (horizonte_anios); las cifras de este informe corresponden a un horizonte de un año (valor por defecto: 1.0 en src/finance.py).

Niveles de cliente y catálogo de incentivos: según su saldo, cada cliente se clasifica como Patrimonial (≥ $120,000), Medio ($50,000 – $120,000) o Básico (< $50,000), y el nivel define qué incentivos puede recibir:

- **Gestor VIP y Tarifa Plana:** $250, efectividad de 45%; solo nivel Patrimonial.
- **Exoneración de comisiones:** $80, efectividad de 35%; niveles Patrimonial y Medio.
- **Bonificación transaccional:** $25, efectividad de 25%; todos los niveles, incluido el Básico.

Para cada cliente se elige el incentivo elegible con mayor ganancia esperada.

Segmento	Condición de Decisión	Valor Patrimonial	Decisión del Sistema	Incentivo Asignado	Costo ($)	Margen Proyectado
A: Alto Valor / Alto Riesgo	$p_i \geq 0.30$ y $E[G] > 0$	Saldo ≥ $120,000	Retención Prioritaria	Exoneración de comisiones (723) / Gestor VIP y Tarifa Plana (135) / Bonificación (1)	$91,615	$289,349
B: Medio / Riesgo Moderado	$p_i \geq 0.30$ y $E[G] > 0$	Saldo < $120,000	Retención Automatizada	Exoneración de comisiones (621) / Bonificación transaccional (255)	$56,055	$182,490

C: No Rentable	$p_i \geq 0.30$ y ($V_i \leq 0$ o $E[G] \leq 0$)	Margen ≤ 0 (446) o incentivo no rentable (101)	Salida Permitida	Sin intervención	$0.00	Sin gasto
D: Estable	$p_i < 0.30$	Cualquier margen	Sin Acción Requerida	Mantenimiento Ordinario	$0.00	Estable

Comparación con la estrategia ingenua: dar el incentivo más barato a todo cliente con probabilidad ≥ 0.5 tiene un ROI mayor (844%) porque gasta poco, pero su ganancia neta es de solo $255,373 y 314 de esos clientes tienen ganancia esperada negativa (−$8,120). La regla propuesta genera $471,839, es decir, un 85% más de ganancia neta. Por eso el criterio de decisión es maximizar la ganancia neta y no el ROI.

En total, el segmento A agrupa 859 clientes y el B 876 (1,735 intervenciones). El segmento C reúne 547 clientes con riesgo de fuga pero sin intervención rentable: 446 son deficitarios (margen ≤ 0) y 101 tienen margen positivo, pero el incentivo cuesta más de lo que se espera rescatar. Su saldo medio es cercano a cero. El segmento D reúne 7,718 clientes estables con probabilidad media de fuga de 9.9%. La plantilla del enunciado sugiere probabilidades ≥ 0.65 para los segmentos A y B; aquí el corte no es fijo, sino que lo determina la ecuación de ganancia neta de cada cliente (probabilidad media de 0.55 en A y 0.57 en B).

4. Operación de la Aplicación en Sucursales

La aplicación, desarrollada en Streamlit (app/app.py), solo hace inferencia: carga los artefactos de la carpeta models/ y no reentrena la red. Tiene cinco pestañas:

- **Buscador:** busca un cliente por CustomerId o apellido y muestra la probabilidad de la red, el margen, la decisión, el incentivo, el cálculo de la fórmula y la explicación SHAP.
- **Cliente nuevo:** formulario con validación de columnas y rangos, y carga masiva por CSV.
- **Panel de cartera:** capital en riesgo, capital rescatable, inversión, ROI, matriz de decisión y lista de clientes prioritarios.

- **Simulador:** permite cambiar costos y efectividades en la barra lateral y comparar contra los valores por defecto y contra la estrategia ingenua.
- **Modelo:** métricas técnicas, curvas y factores de abandono.

Las siguientes capturas muestran las cuatro vistas de trabajo de la aplicación (la pestaña Modelo reproduce las Figuras 1 y 2).

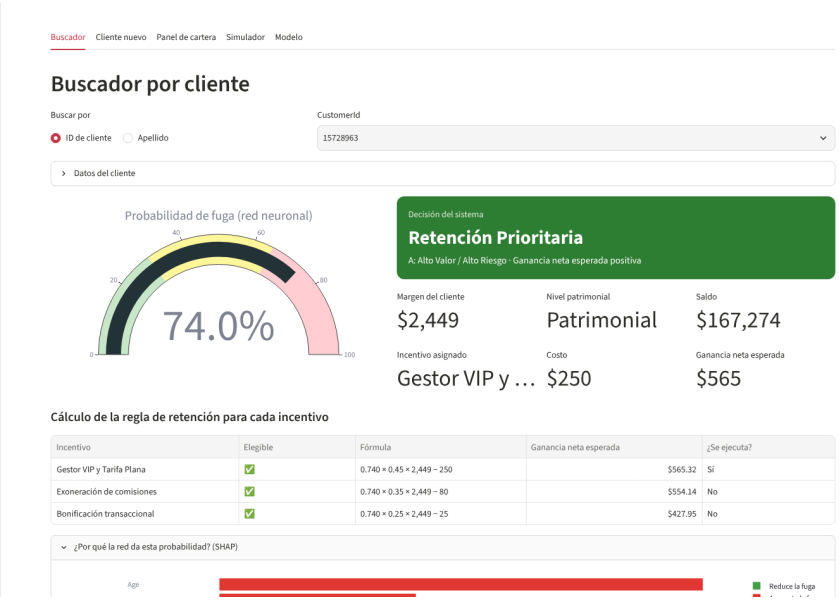


Figura 3. Pestaña Buscador: probabilidad de fuga de la red, decisión del sistema (Retención Prioritaria), margen, incentivo asignado y cálculo de la regla para cada incentivo.

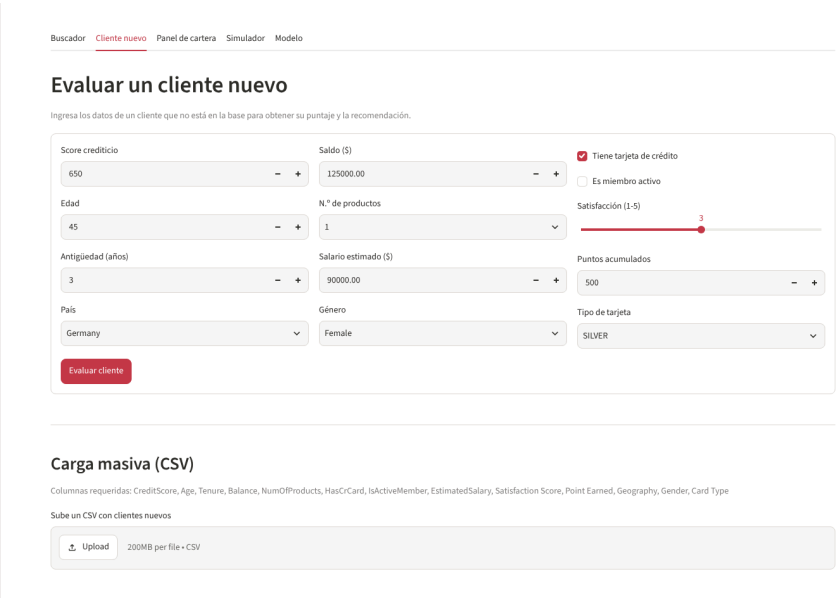


Figura 4. Pestaña Cliente nuevo: formulario con validación de datos y carga masiva por CSV.

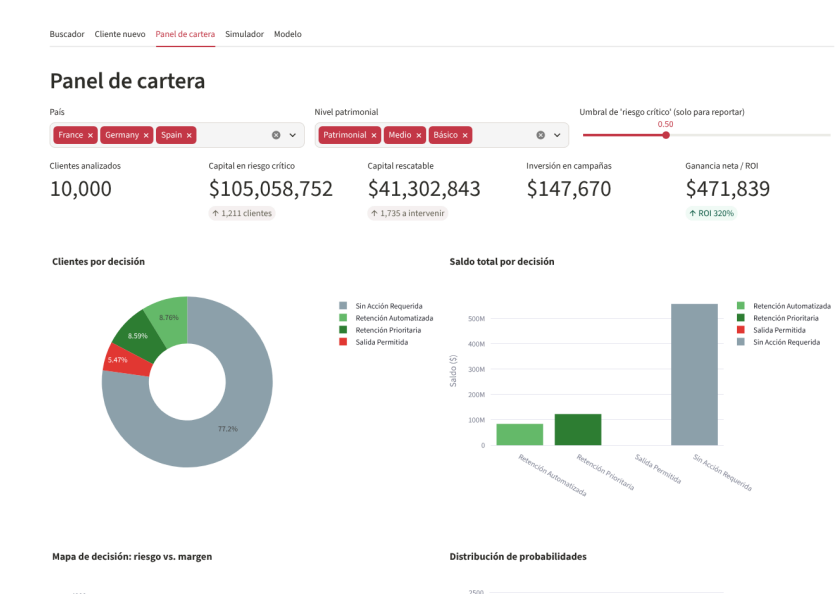


Figura 5. Pestaña Panel de cartera: clientes analizados, capital en riesgo crítico, capital rescatable, inversión, ganancia neta y ROI, con distribución de clientes y saldo por decisión.

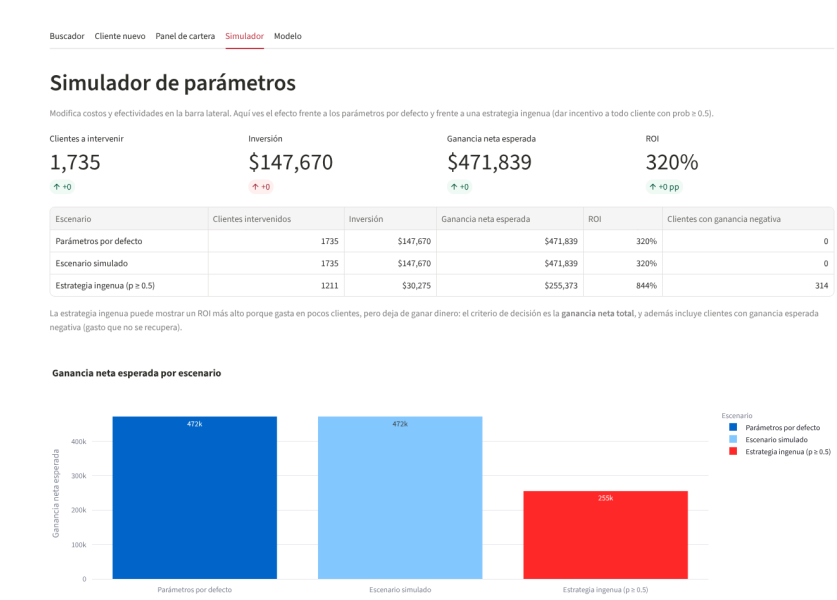


Figura 6. Pestaña Simulador: efecto de cambiar costos y efectividades en la ganancia neta y el ROI, comparado con la estrategia ingenua.

Procedimiento del Gestor de Cuentas:

1. Acceder a la pestaña de "Buscador" dentro de la aplicación.
2. Ingresar el identificador del cliente para consultar su perfil.
3. Analizar la interfaz de resultados que visualiza el puntaje de fuga emitido por la red neuronal, el impacto monetario proyectado y la decisión comercial sugerida.
4. Ejecutar estrictamente la acción recomendada por el sistema según el catálogo de acciones.

Mantenimiento del Modelo:

Para garantizar que el sistema "Deep Retention" mantenga su precisión y no sufra degradación por cambios en el comportamiento transaccional o condiciones macroeconómicas (Data Drift), se establece el siguiente protocolo de mantenimiento basado en la arquitectura modular del proyecto:

- **Frecuencia de Actualización de Datos:** Los lotes de datos transaccionales y patrimoniales de los clientes deben extraerse y consolidarse de forma mensual. El archivo de ingesta (similar al esquema de Customer-Churn-Records.csv) se depositará en el directorio data/ para su procesamiento automatizado mediante src/preprocessing.py.
- **Monitoreo y Reentrenamiento:** El equipo de Inteligencia de Negocios evaluará las métricas del modelo (almacenadas en reports/metrics.json) de forma trimestral. Si el indicador ROC-AUC cae por debajo de 0.80 o el Brier Score aumenta de forma significativa respecto a la versión vigente, se ejecutará el pipeline src/train.py para reentrenar la red neuronal.
- **Control de Versiones de Pesos:** Tras cada reentrenamiento exitoso que supere el umbral de métricas de diagnóstico, se generará una nueva versión de los archivos deep_retention_mlp.keras, preprocessor.joblib y calibrator.joblib en el directorio models/, actualizando automáticamente el motor de la aplicación en producción.

5. Conclusiones y Próximos Pasos

Conclusiones

El sistema "Deep Retention" muestra que una red neuronal combinada con una regla financiera explícita permite dejar atrás la retención indiscriminada y pasar a una retención selectiva y rentable. El modelo alcanzó un ROC-AUC de **0.851** sin usar variables con fuga de datos, y la calibración de las probabilidades permite usarlas directamente en el cálculo monetario.

En lo financiero, el sistema recomienda una inversión de **$147,670** en campañas de fidelización, con un ROI estimado de **320%** y un capital rescatable de **$41.30 millones**. Además, clasifica como "Salida Permitida" a los clientes deficitarios y evita gastar en ellos.

Frente a la estrategia ingenua (incentivar a todo cliente con probabilidad ≥ 0.5), el enfoque propuesto genera una ganancia neta mayor ($471,839 frente a $255,373), porque solo interviene donde la ganancia esperada es positiva.

Los factores de abandono identificados con SHAP (edad, número de productos, nivel de actividad y país) indican que la retención debe apoyarse en la vinculación de productos y en reactivar a los clientes inactivos.

Una limitación es que el dataset no incluye comisiones ni volumen transaccional, así que el margen de cada cliente es una estimación basada en supuestos ajustables. Por eso, la validación en un entorno real es el siguiente paso necesario.

Consideraciones éticas

El modelo usa variables sensibles como Gender y Geography, y el análisis SHAP muestra que influyen en la probabilidad estimada. Para mitigar el riesgo de trato desigual, el incentivo se asigna por la rentabilidad esperada del cliente y no por su perfil demográfico, la decisión es transparente para el gestor (fórmula y explicación SHAP visibles) y la variable Complain se excluyó para no usar información posterior a la salida. Durante el piloto se recomienda monitorear las tasas de intervención y de error por género y país, y evaluar si retirar estas variables afecta de forma relevante el desempeño.

Próximos pasos: prueba piloto y despliegue

Fase	Duración	Actividades	Criterio de éxito
1. Preparación	Semanas 1–2	Integrar datos reales de comisiones y transacciones para recalcular el margen. Capacitar a los gestores en el uso del Buscador.	Datos validados y gestores capacitados
2. Piloto controlado	Semanas 3–14 (3 meses)	Escoger 2–3 sucursales. Dividir a los clientes en un grupo tratado (sigue las decisiones del sistema) y un	Menor tasa de fuga en el grupo tratado y ROI real positivo

		grupo de control (gestión tradicional).	
3. Evaluación	Semanas 15–16	Comparar la fuga real con la predicha, la efectividad real de cada incentivo y el ROI. Ajustar los costos y las efectividades en el simulador.	Calibración estable y parámetros financieros actualizados
4. Despliegue general	Meses 5–6	Llevar el sistema a toda la red de sucursales de forma escalonada por región, con reentrenamiento trimestral según el protocolo de mantenimiento.	Adopción total y ROC-AUC ≥ 0.80

Más adelante se recomienda reemplazar las efectividades supuestas de los incentivos por valores medidos en el piloto y explorar modelos de *uplift*. Estos modelos estiman directamente cuánto reduce cada incentivo la probabilidad de fuga.

6. Anexo: Reproducibilidad

Stack tecnológico: Python 3.12, TensorFlow/Keras, scikit-learn, SHAP, pandas, NumPy, Streamlit, Plotly, Matplotlib y joblib.

El repositorio separa el entrenamiento de la inferencia:

- **app/app.py:** aplicación Streamlit (solo inferencia).
- **data/:** Customer-Churn-Records.csv.
- **models/:** deep_retention_mlp.keras (pesos), preprocessor.joblib, calibrator.joblib y shap_background.npy.
- **reports/:** metrics.json, resumen_financiero.json, cartera_puntuada.csv, matriz_decision.csv, importancia y gráficos SHAP, y diagnóstico del modelo.
- **src/config.py:** rutas, columnas, semilla e hiperparámetros.

- **src/preprocessing.py:** carga, validación, escalado y codificación.
- **src/model.py:** arquitectura de la red neuronal.
- **src/train.py:** canal de entrenamiento, calibración, evaluación, SHAP y puntuación de la cartera.
- **src/predict.py:** canal de inferencia y explicabilidad SHAP.
- **src/finance.py:** margen, regla de ganancia neta, segmentación, catálogo y estrategia ingenua.

Dependencias (requirements.txt): pandas ≥ 2.2, numpy ≥ 1.26, scikit-learn ≥ 1.5, tensorflow ≥ 2.16, streamlit ≥ 1.50, shap ≥ 0.46, plotly ≥ 5.20, matplotlib ≥ 3.8 y joblib ≥ 1.3.

Ejecución: (1) pip install -r requirements.txt; (2) opcional, reentrenar con python -m src.train (los pesos ya vienen en models/); (3) streamlit run app/app.py. La semilla fija definida en config.py garantiza resultados reproducibles. Repositorio: github.com/Patriciadocasar/Proyecto1IA.